

Figure 1:

FACULTAD DE CIENCIAS

MATERIA: ICC

Escuela: UNAM

Número de cuenta **324181883**
Nombre del alumnno: **Melissa Prado Hernandez**
Número de practica: **Prcatica05**
Fecha de entrega: **23 de Octubre del 2026**

Melissa Prado Hernandez

23 de October 2026

1 Parte 1: Calculando Pi a dardazos

Para esta primera parte de la practica nos dan una explicación general de como resolver el problema.
Lo primero que hice fue definir que tipos de variables necesitaba ocupar para cada dato. Por ejemplo para el tipo de dato de 1000.0 para saber con cuantos dardos en totales tenemos, use una variable de tipo double, para que en las siguientes operaciones no tuviera problema la variable con los puntos decimales. Tambien inicie una variable con un identificador de dardosdentrodeElcirculo con un valor de 0.0 porque este se encarga de contar cuantos dardos van a entrar dentro del circulo. Por último solo declare mi contador para mi ciclo del while i es igual a 0, este practicamente se utiliza para que te diga cuantos lanzamientos se intentaron independientemente si entraron o no.
Para mi ciclo en while, asigne una condición como fue descrita en la prática, que mi contador (Lanzamientos) fuera menos al numero de dardos totales para que se repita el ciclo hasta que llegue al total de datos declarados y parar. Dentro del cuerpo del while declare las variables que se iban a ocupar para poder sacar la distancia con la fomula, para esto fue necesario utilizar Math.random() porque te estrega un valor aleatorio entre 0 y 1. Ademaás, le apliqué Math.abs() porque no queremos numeros negativos ya que son distancias, entonces ocupamos valor absoluto para evitarnos ese problema.
Ahora dentro del mismo del cuerpo del while, solo apliqué la formula de las distancias con la herramienta de elevar al cuadrado de java Math.pow() esta operación la guarde en una variable de nombre distancias ya que lo ibamos a ocupar despues en una condición.
Para hacer una condición vamos ocupar if, en este caso nuestra condición nos la habían dado en la práctica la distancia debia ser menor o igual a 1, con el proposito de verificar si el dardo cayó dentro del circulo. Si se cumplia la condición, se aunmentaba el contador dardosdentrodeElcirculo++ con el operador de incremento, este funciona como nuestra actualización para el while.
En esta parte solo coloqué el operador de incremento de nuestro contador de lanzamientos i++ pero fuera del bloque if, este lo utilicé afuera debido a que independiente si entraban los tiros o no tenia que contar cuantos lanzamientos fueron en total.
En la siguiente parrte de mi código, solamente se va calcular la aproximación de PI con la fomula que nos dieron en la práctica, se dividia los dardos dentro

del circulo entre el total de dardos y ese resultado se multiplicaba por 4.0 por el despeje matematico.Ṕor último, simplemente voy a impirmir los resultados para que el usuario pueda verlos. Algo para destacar es que nos pedian 6 digitos de PI en nuetro código, entonces ocupé System.out.prinf con su formato correspodiente para poder imprimir la estimación exacta de PI.

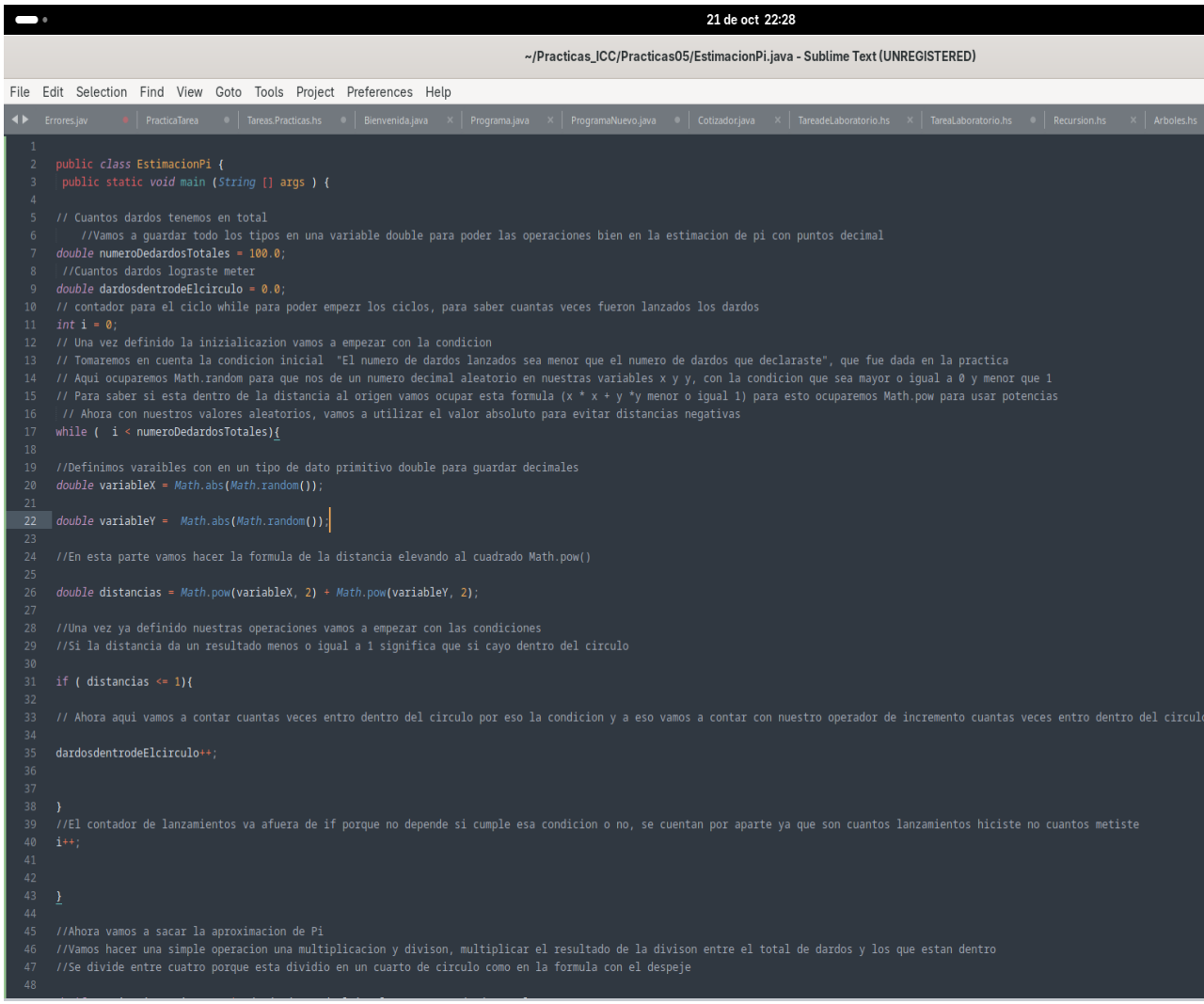


Figure 2: Código de Estimación de PI

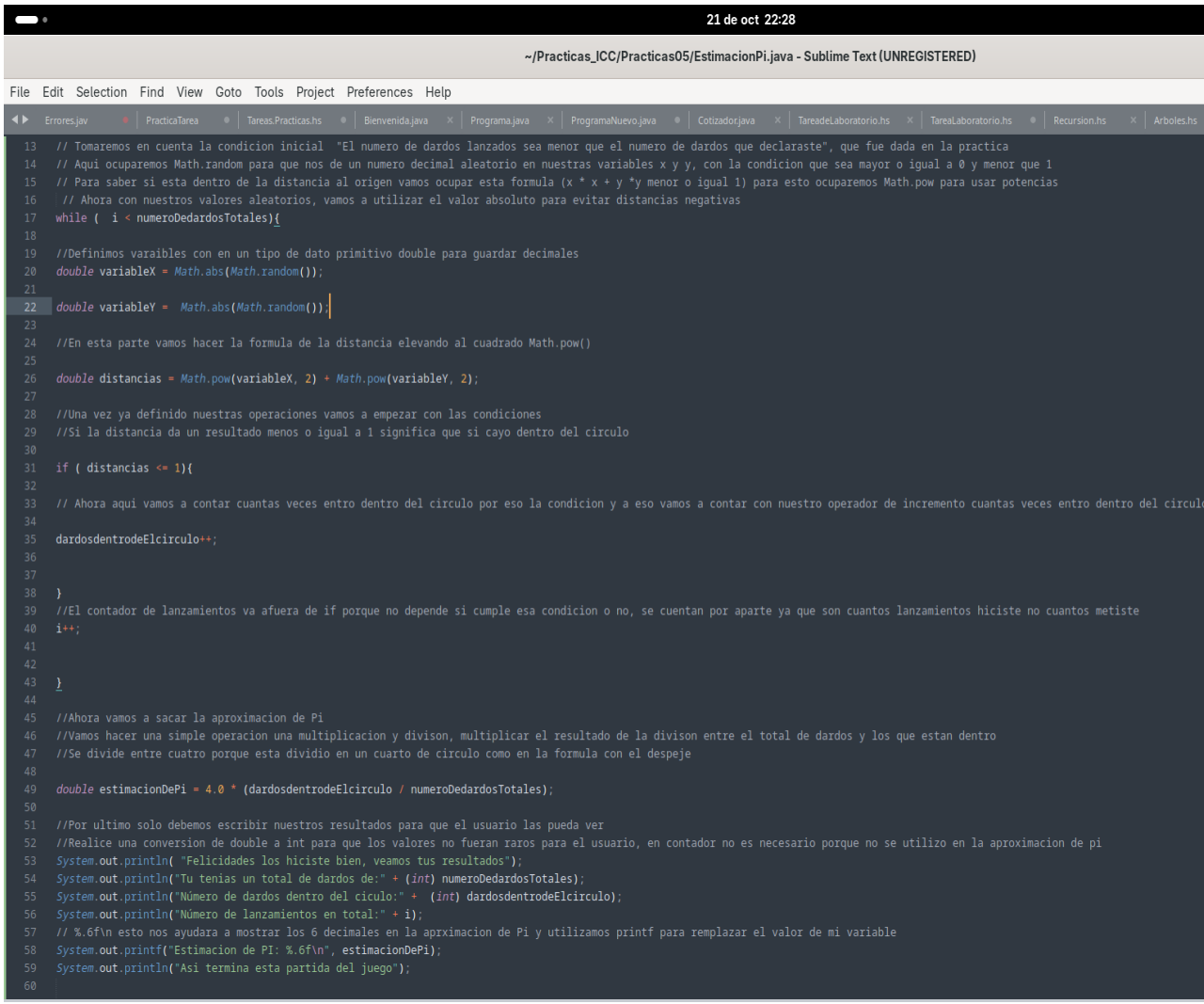


Figure 3: Código de Estimación de PI

Compilación y ejecución del código

Con los difrentes valores de dardos totales para probar cada caso (100,1000,10000,100000 y 1000000)

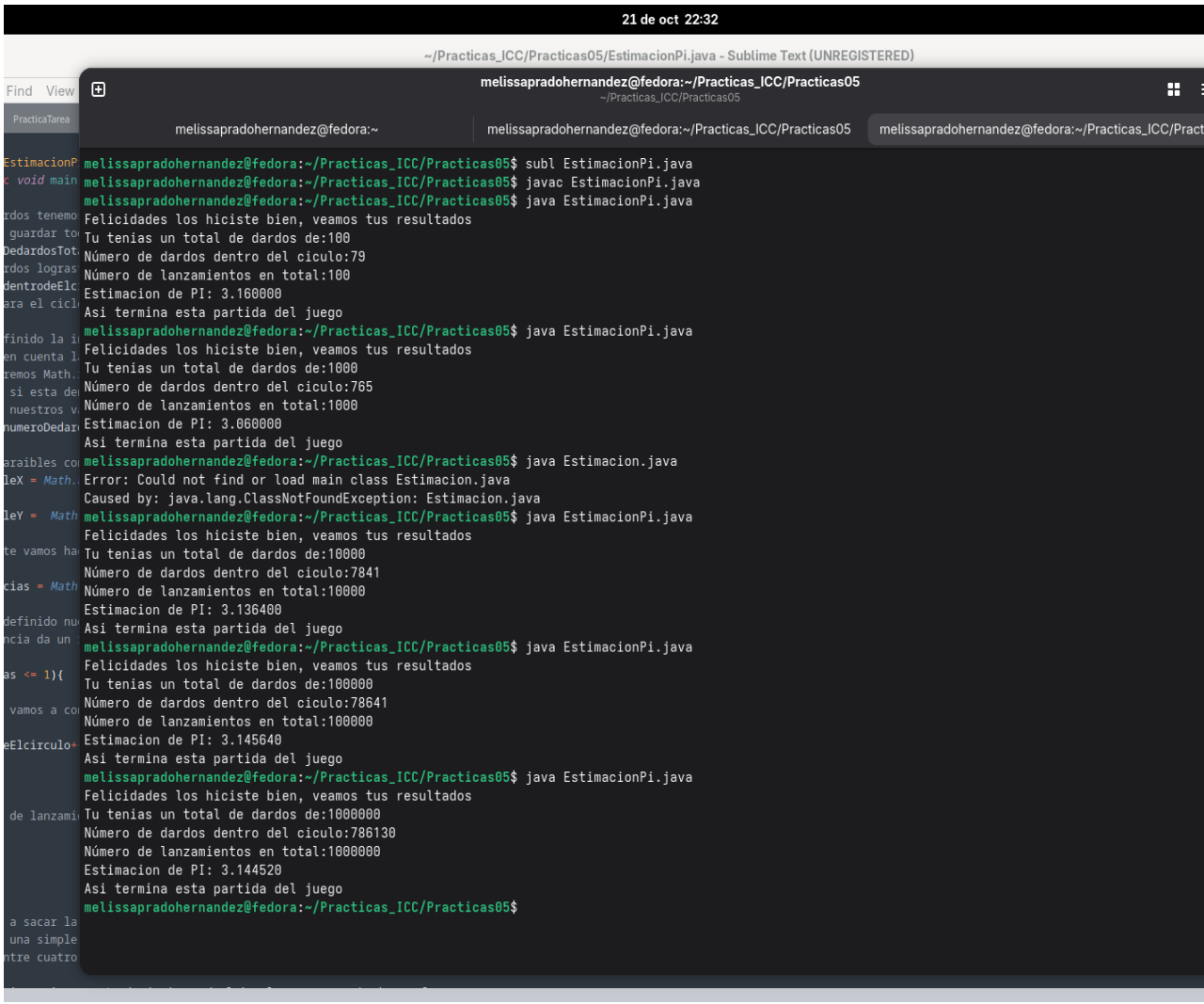


Figure 4: Compilación y ejecución

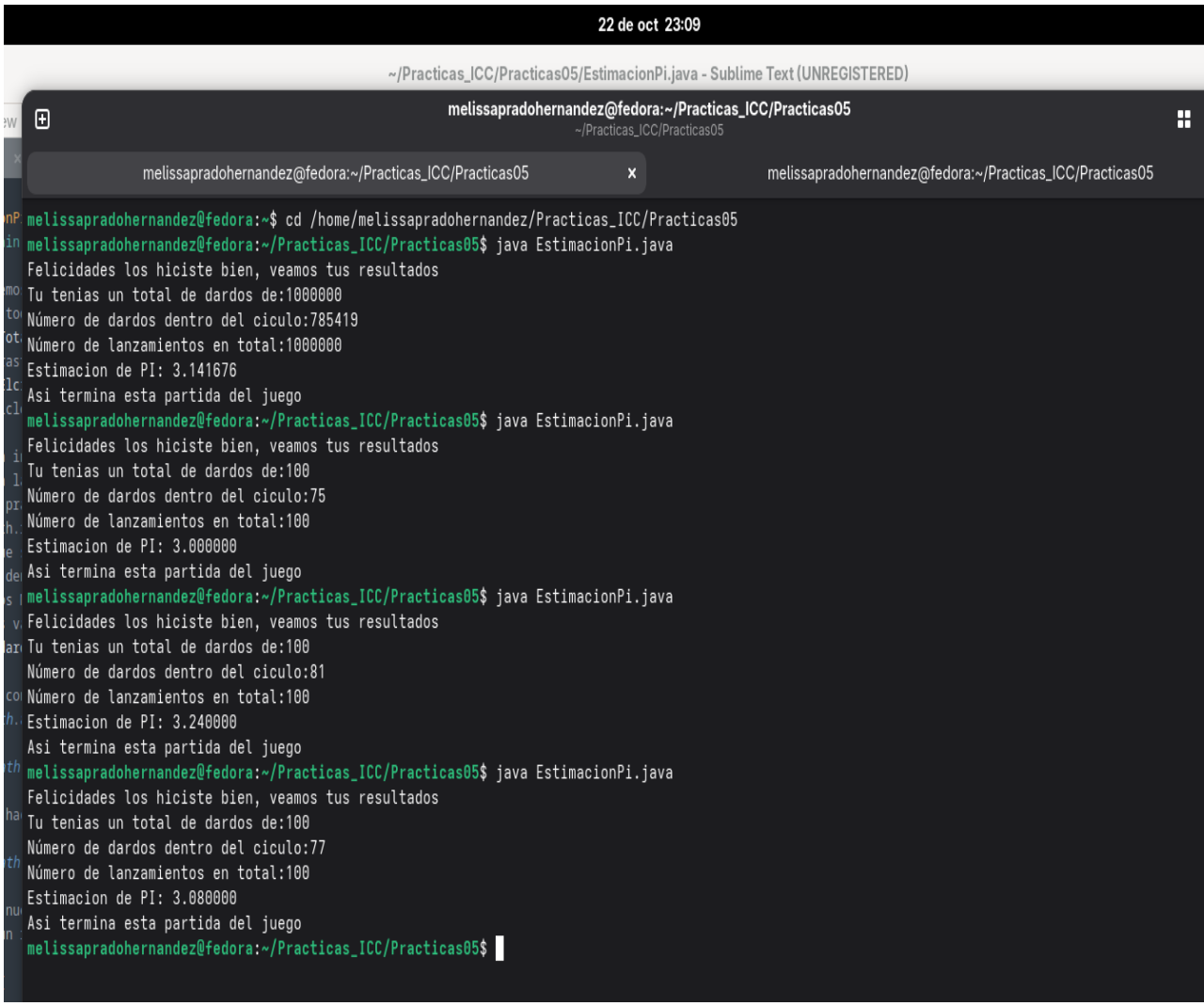


Figure 5: Compilación y ejecución de el mismo número con diferencia de decimas

Tabla completa de valores

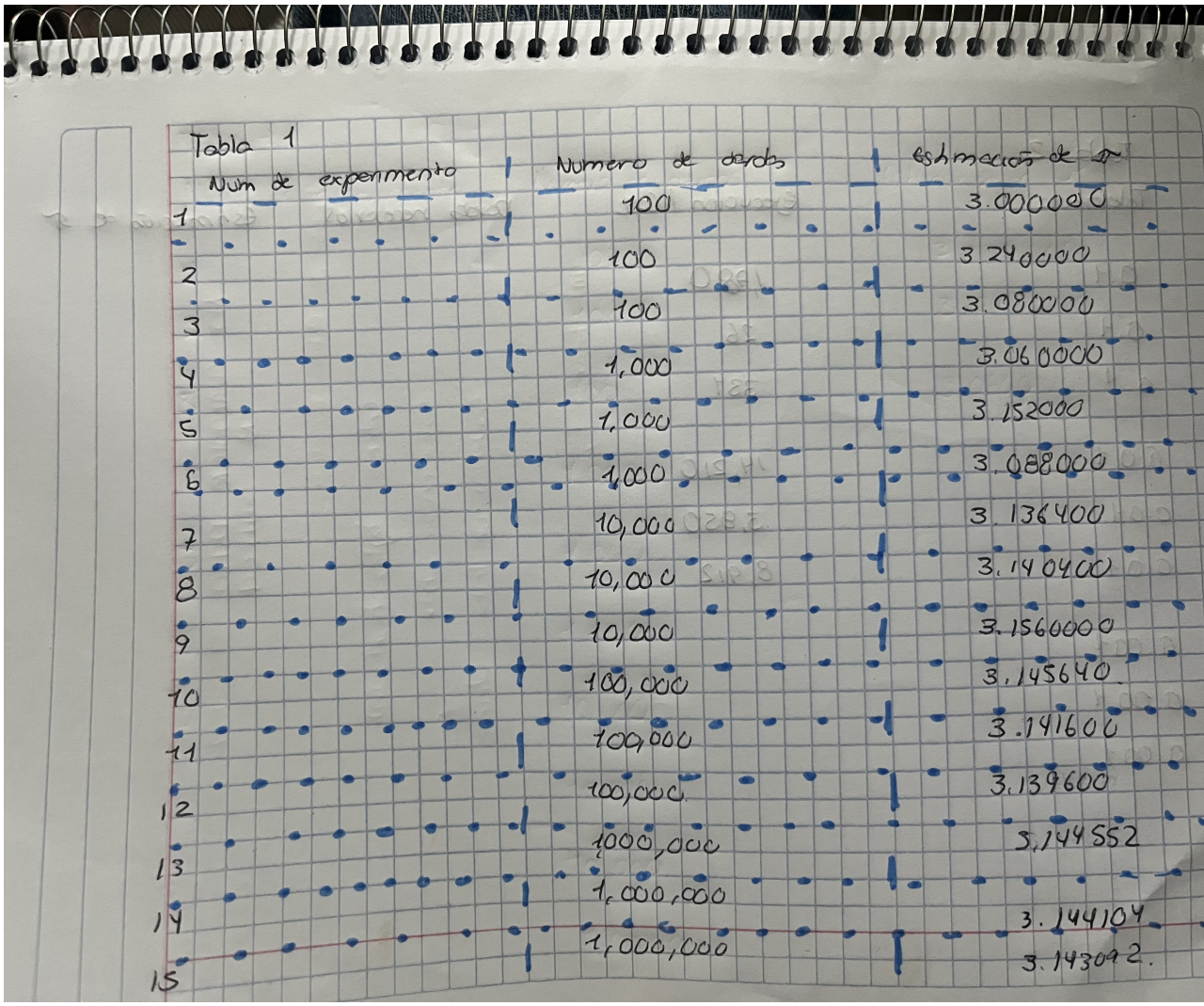


Figure 6: Tabla de valores

¿Por qué tres ejecuciones con el mismo número de dardos dan resultados distintos ?
Por que recordando que cada vez que el programa corre, la funcion Math.random genera un numeros aleatorios en los pares ordenados. Por eso al cambiar los puntos generados en cada ejecución. Por eso cammbia la estimación de pi ligeramente.
¿Qué pasa con la estimación conforme aumenta el número de dardos?
Mientras el número de dardos este en aunmento, la estimación de pi se va acercando a su valor original con la función Math.PI.

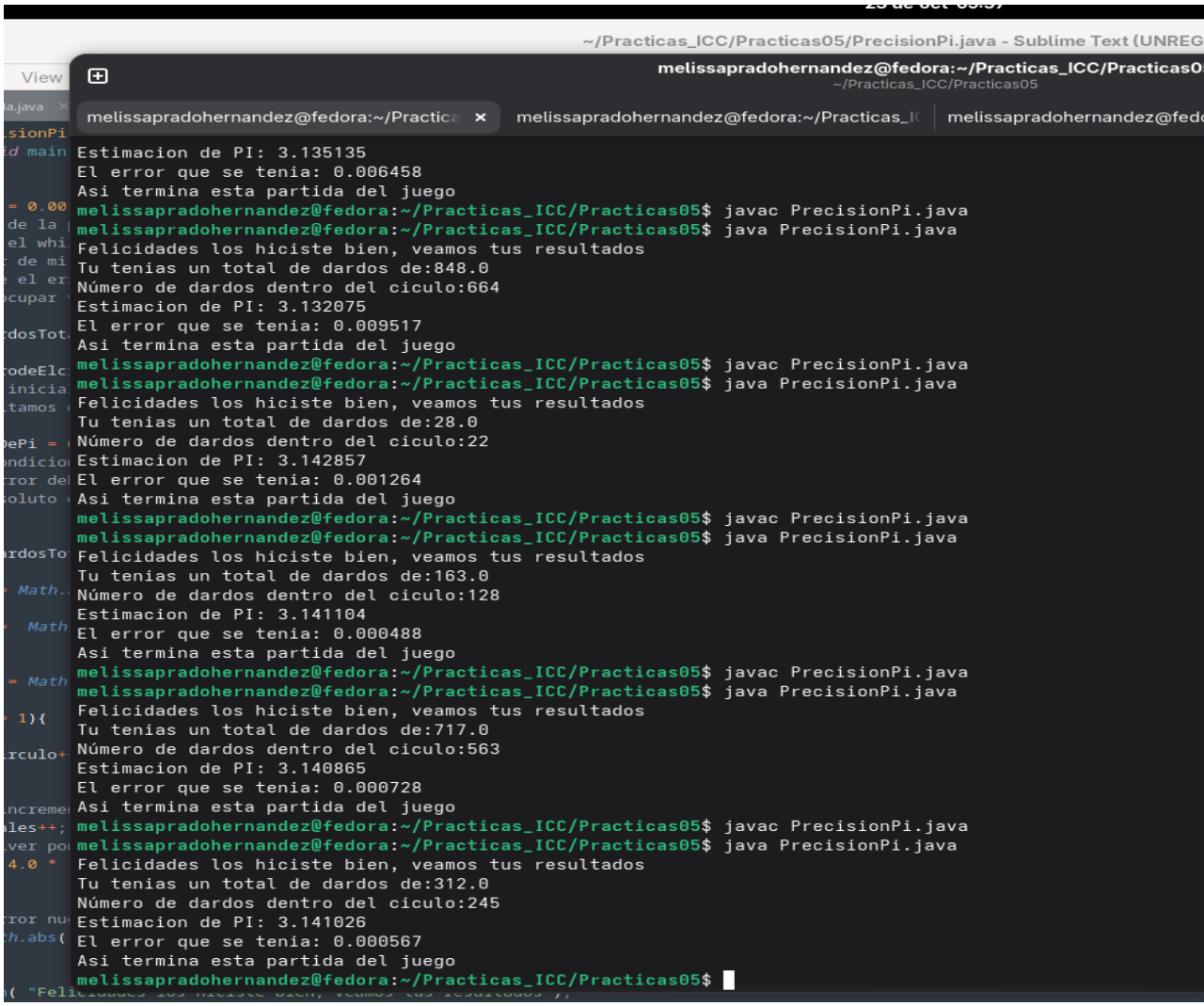


Figure 7: Compilación y ejecución de el mismo número con diferencia de decimas

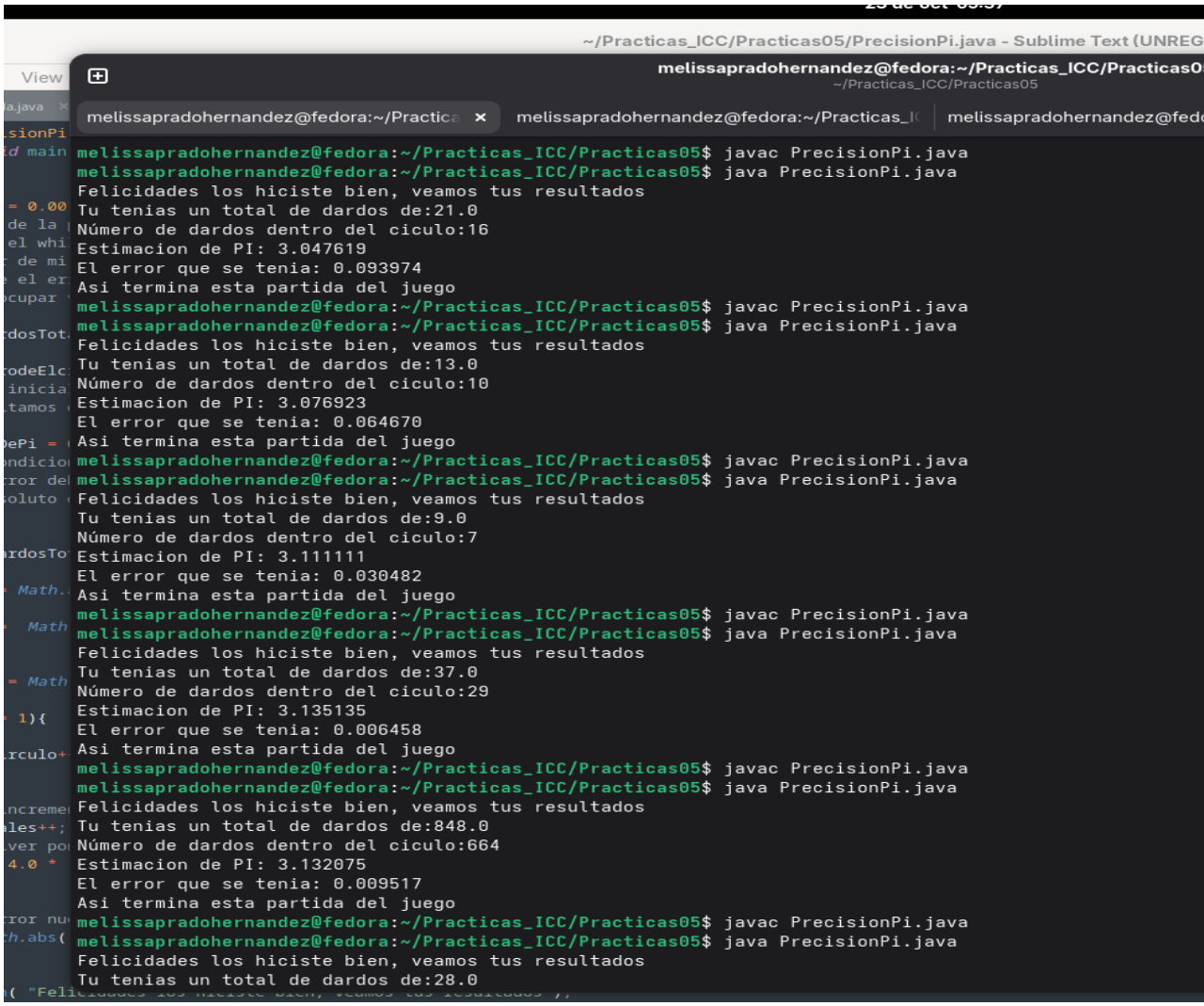


Figure 8: Compilación y ejecución de el mismo número con diferencia de decimas

2 Actividad 1.2: Repetir hasta llegar a la precisión deseada

Para empezar definí la variable de nombre tolerancia como tipo double (probando con los valores 0.1, 0.01, 0.001. Anteriormente en la Actividad 1.1 ya habia definido las demas variables. Para poder iniciar con mi while primero inicialicé la variable estimacionDePi = 0.0.
Para mi condición, lo primero es definir que el programa al menos lance un

dardo para que no tenga errores numeroDedardosTotales == 0, ahora en while utilizé Math.abs(estimacionDePi-Math.Pi todo eso mayor o igual a la tolerancia. Después lo que se realizo fue sacar su valor absoluto de esa operación para evitar números negativos. Este ciclo se va ejecutar si la diferencia de entre mi aproximación y PI sea mayor o igual a tolerancia.
Adentro del ciclo, en cada vuelta generé las coordenadas x y y con Math.abs(Math.random()). Después calculé la distancia al origen elevando cada variable al cuadrado con Math.pow(x, 2) + Math.pow(y, 2). Si la distancia daba menor o igual a 1, aumentaba mi contador dardosdentrodeElcirculo++ con el operador de incremento, este inmediatamente después, fuera del if iba ir sumando numeroDedardosTotales++ para registrar que ya se había realizado un intento completo.
Al final de cada vuelta dentro del while, vuelvo a calcular estimacionDePi = 4.0 * (dardosdentrodeElcirculo / numeroDedardosTotales). Asigné el valor a la variable que ya estaba declarada arriba para que en la siguiente vuelta del while Java evalúe la condición con el dato actualizado y no se quede en un ciclo infinito.
Afuera del ciclo, vuelvo a calcular double error = Math.abs(estimacionDePi - Math.PI); para poder imprimirlo en pantalla, ya que el calculado dentro del while no se puede usar directamente fuera. Por último, imprimo los resultados para que el usuario pueda verlos, asi como en la anterior práctica utilice prinf para poder impimir los 6 decimales exactos .
Estuve en el laboratorio resolviendo la practica

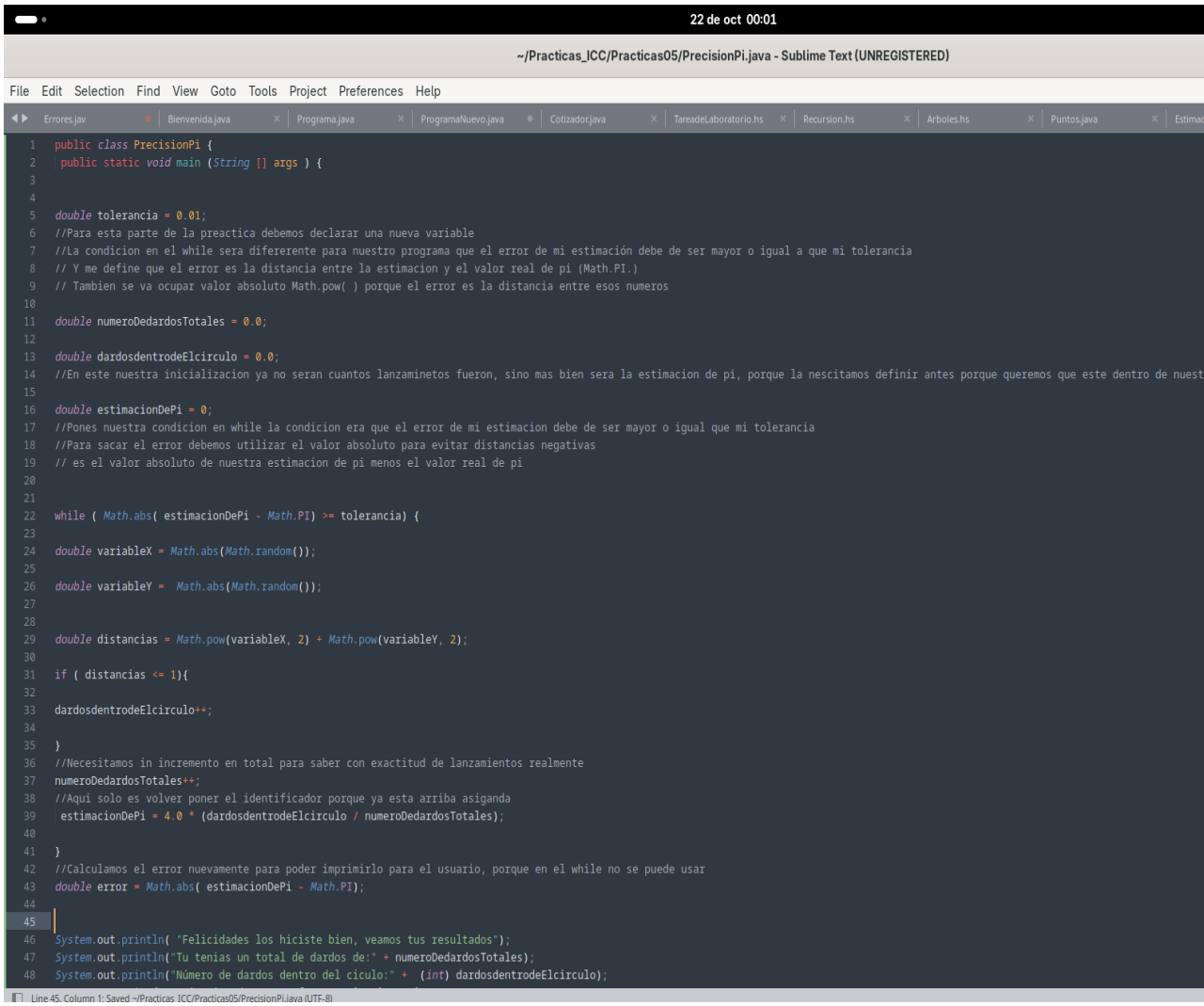


Figure 9: Código Precision de Pi

Validación de valores negativos o cero: Para validar que no diera errores ni valores negativos, utilicé un if inicial con el operador or. Puse las condiciones preciodelCliente1 ¡= 0 or plazodelCliente1 ¡= 0 or interesAnual ¡= 0. Si alguna de estas se cumplía, el programa mostraba un mensaje advirtiendo que los valores no debían ser negativos ni cero.

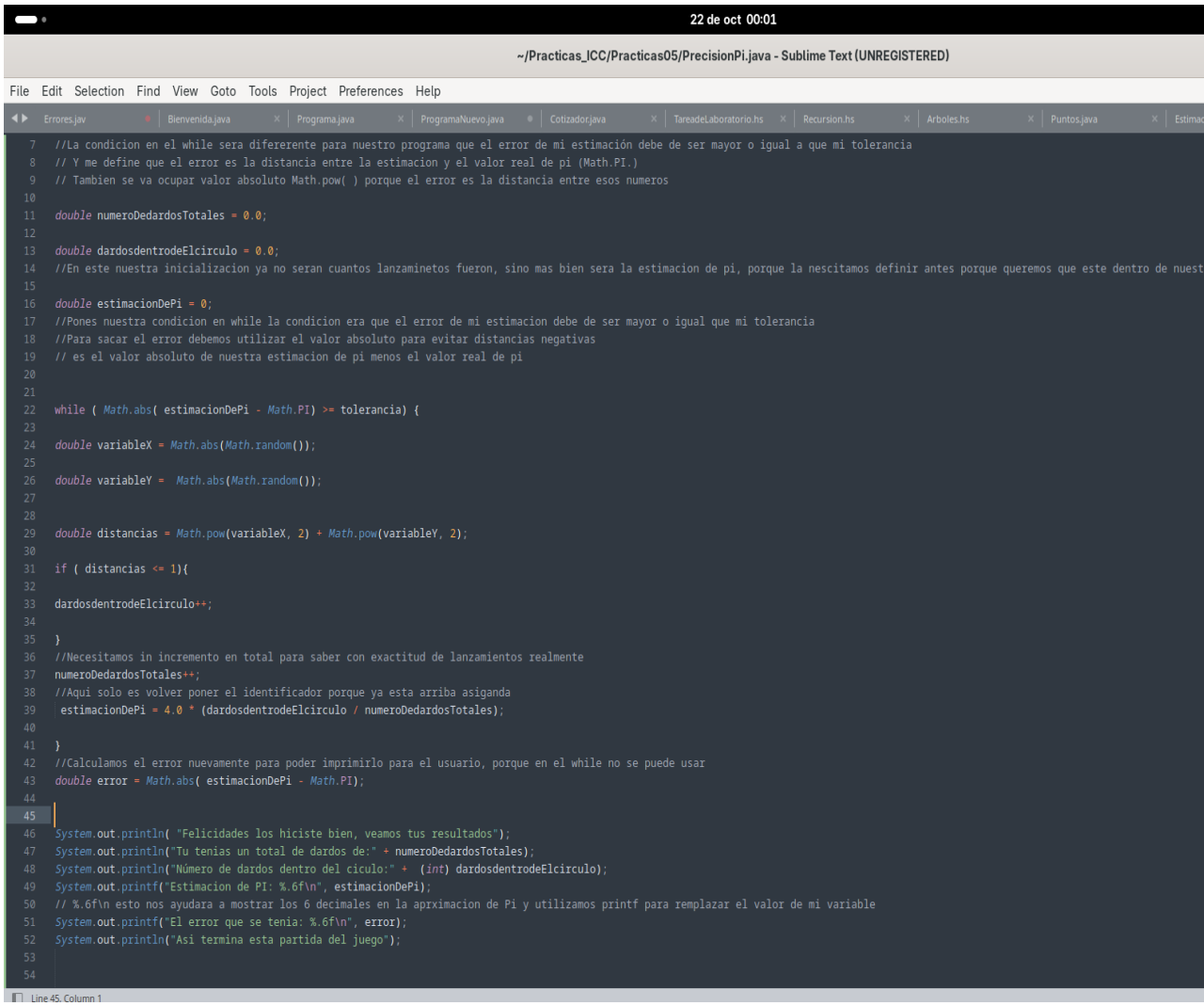


Figure 10: Código de Precision de Pi

Compilación y ejecución del Programa Precision a Pi

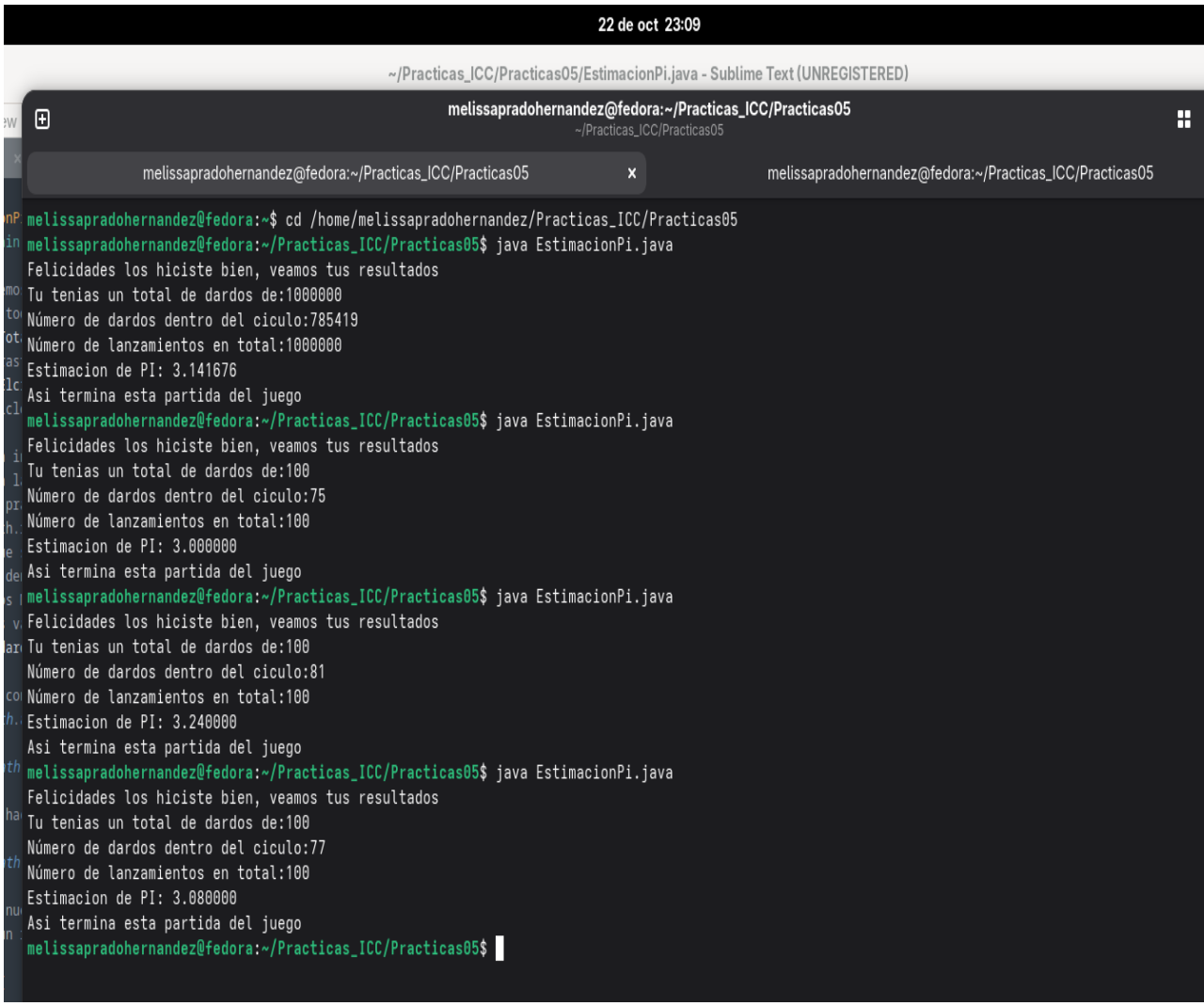


Figure 11: Compilación y ejecución de Precision de Pi

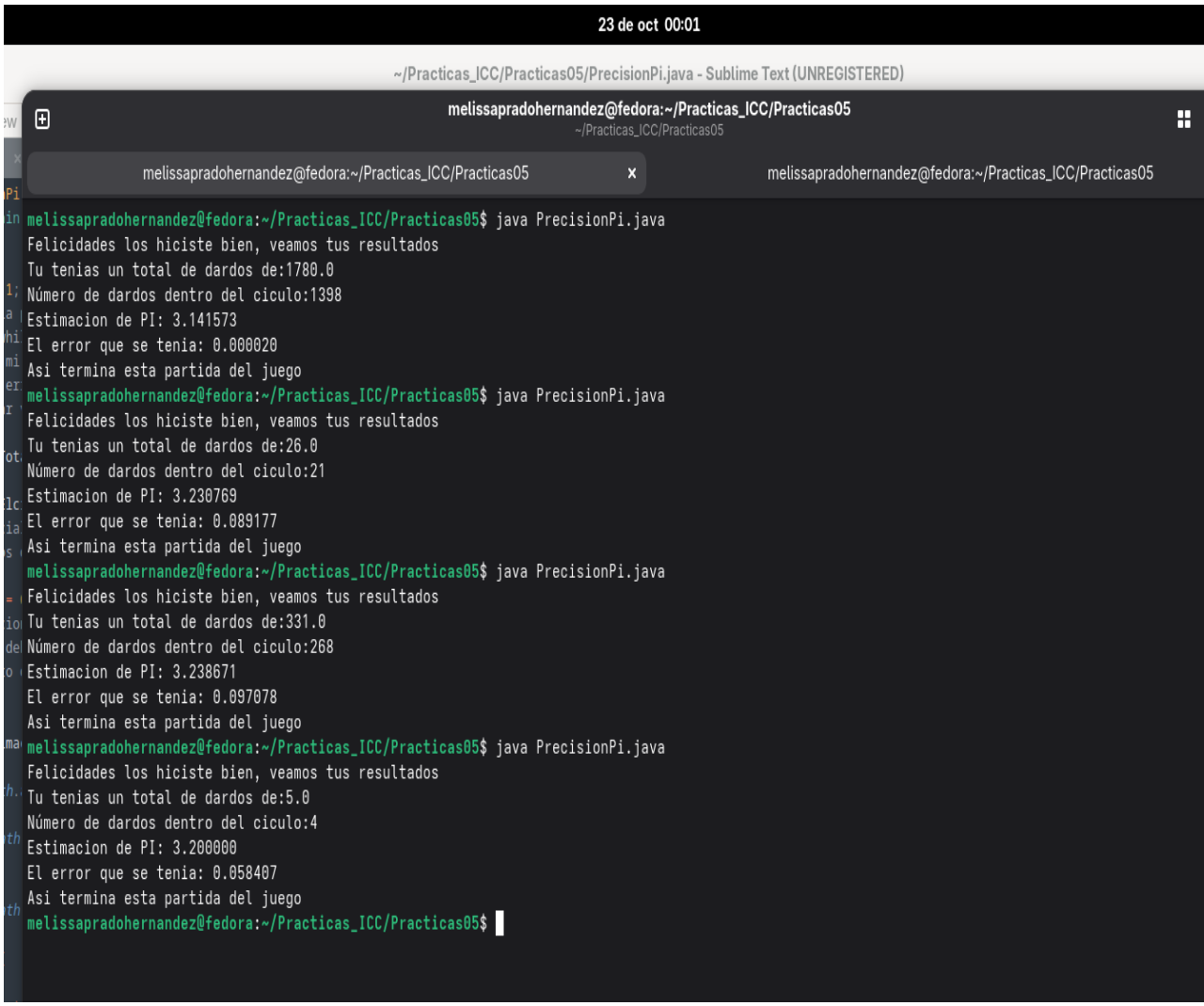


Figure 12: Compilación y ejecución de Precision de Pi en 0.1

Tabla de los valores

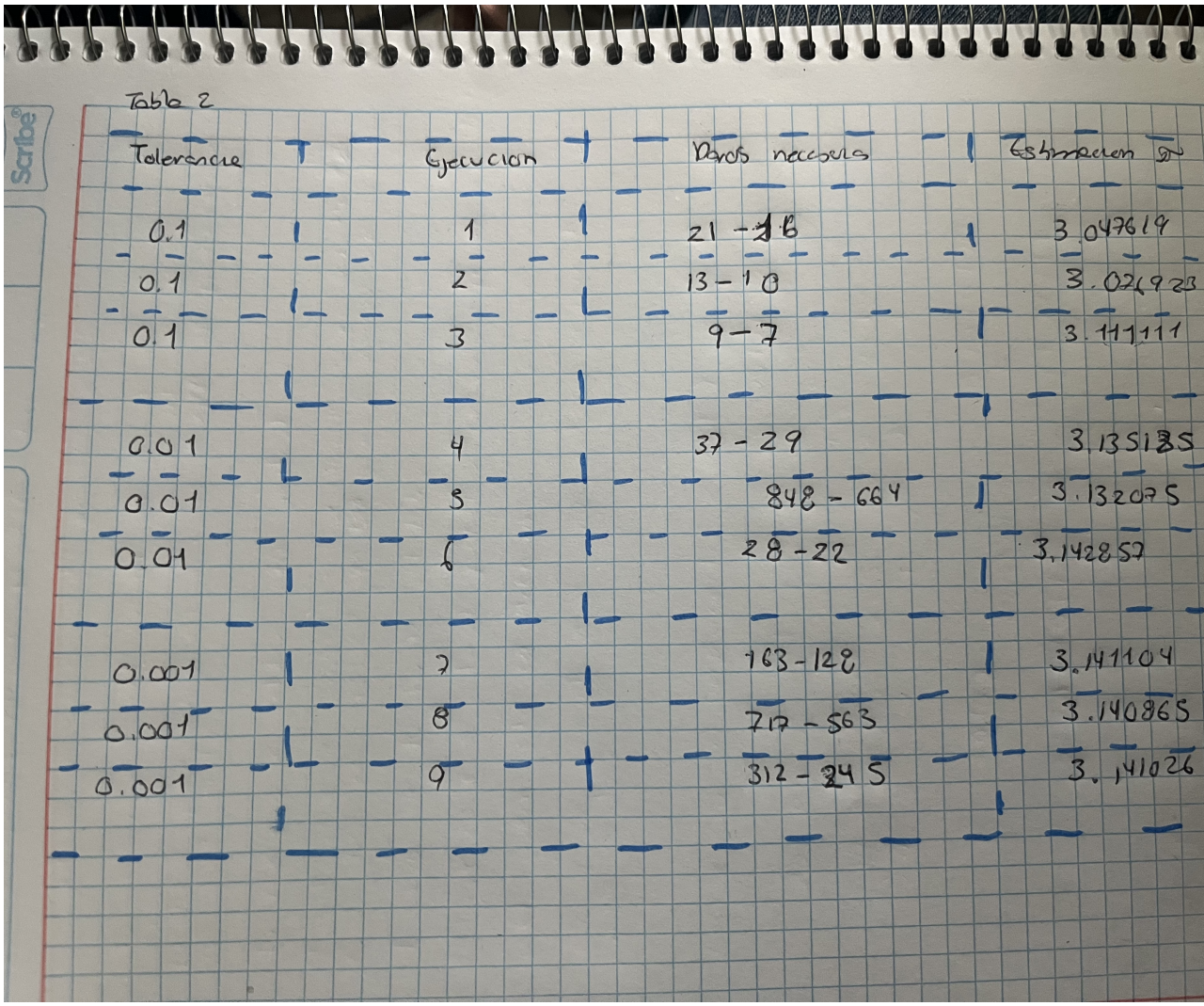


Figure 13: Tabla de valores

¿Qué tanto variaron los datos los dardos necesarios entre las tres ejecuciones de una misma tolerancia? ¿Te parece que el promedio respresenta bien lo que paso?
Siento que el promedio no representa muy bien lo que pasó, porque al ser solo 3 intentos, un número muy alto cuando en realidad dos de las tres corridas terminaron con menos de 40 dardos.

Compara esta tabla con la de la actividad 1.1 ¿Cuántos dardos necesitaste para llegar a un error menor a 0.001? ¿Concuerda con lo

que observaste con un número fijo de dardos?. Si no concuerda, ¿por qu e crees que pasa? Como pequeña pista, piensa en qué momento exacto se detiene tu ciclo.
En la Actividad 1.2 logré llegar a un error menor a 0.001 con solo 397 dardos en promedio. En cambio, en la Actividad 1.1 con lanzamientos fijos, se necesitaron de 100,000 a 1,000,000 de dardos paraque esa misma precisión de forma constante. Esta diferencia ocurre por que while termina de forma diferente en la Actividad 1.2, el programa evalua el error en cada dardo que lanza, a diferencia de como el calculo cambia, en el instante en que le pega de pura suerte a un valor muy cercano a Pi, el ciclo se interrumpe de inmediato guardando un número bajo de lanzamientos, mientras que en la Actividad 1.1 el programa lanzar la totalidad de los dardos programados sin detenerse antes, demostrando que para lograr una precisión más cercana a Pi se requieren muchas ejecuciones
.

3 Parte 2: El regreso del Señor Pines

Para empezar definíqur tipo de datos son los del préstamo de la práctica anterior, como el nombre del cliente nombreDelprimerCliente, el precio preciodelCliente1 = 12899, la tasa inicial de interesAnual = 0.15 y el plazo convertido a años plazodelCliente1 = 21.0 / 12.0. Para clasificar los tipos de cliente, declaré la variable char tipoDeCliente = 'P'; (probando tambien con 'E' y 'R'.
Para validar que no diera errores ni valores negativos, utilicé un if inicial con el operador or. Puse las condiciones preciodelCliente1 menor o igual 0 or plazodelCliente1 menor o igual 0 or interesAnual menor o igual 0. Si alguna de estas se cumplía, el programa mostraba un mensaje advirtiendo que los valores no debían ser negativos ni cero.
Usé otra condicion porque necesitaba verificar que el tipo de cliente fuera valido. Utilicé un else if con el operador and (tipoDeCliente diferente o igual 'P' and tipoDeCliente diferente o igual 'E' and tipoDeCliente diferente o igual 'R'). Con esto me aseguré de que si el usuario ingresaba una letra diferente a P, E o R, el programa imprimiera un mensaje de error indicando que solo se podía acceder con esos tipos de clientes.
En el bloque else, evalué las demás posibilidades que no fueron descartadas antes. En este caso lo que hice fue evaluar a cada cliente por casos mediante estructuras if para ver si el cliente valía P, E o R
Caso Cliente P: Si tipoDeCliente =='P', le resté los 2 puntos a la tasa como fue establecido, quedando tasaAplicada = 0.13. Realicé las operaciones de la práctica calculando el interes (interes1), el total (totalResultados1) y la mensualidad dividiendo entre 21 (plazo1), e imprimí los datos para el cliente P.
Caso Cliente E: Si tipoDeCliente == 'E', la tasa se quedó igual con tasaAplicado2 = 0.15. Calculé las tres operaciones correspondientes (interes2, totalResultados2 y plazo2) e imprimí los resultados en pantalla.
Caso Cliente R: Si tipoDeCliente == 'R', le aumenté 3 puntos al valor inicial de la tasa para tener tasaAplicado3 = 0.18. Realicé las operaciones (interes3,

totalResultados3 y plazo3) y mostré las salidas del cliente R. Por último por cada caso se imprimio los datos de cada tipo de cliente que le correspondian. Estuve en el laboratorio resolviendo la práctica

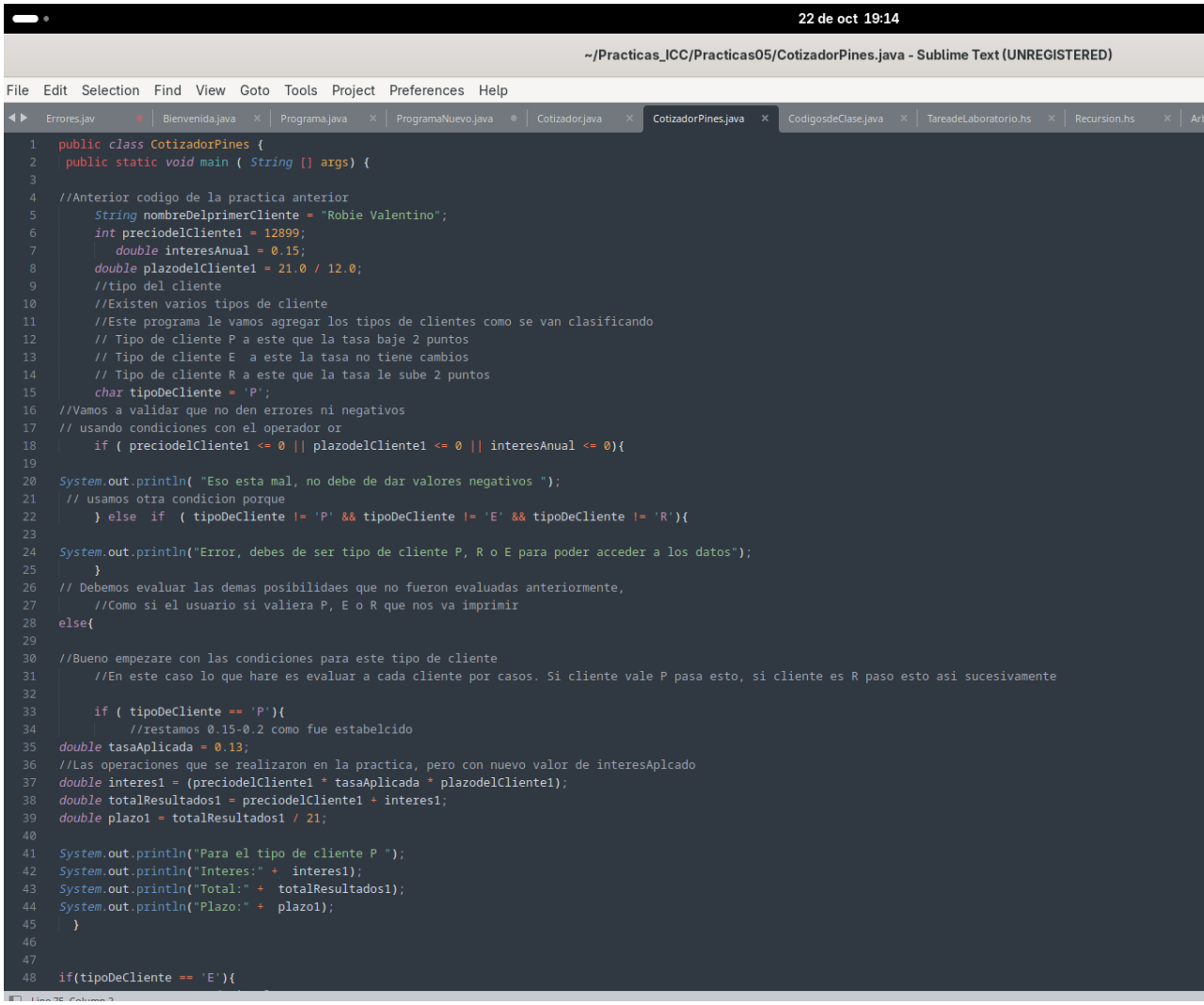


Figure 14: Código del Señor Pines

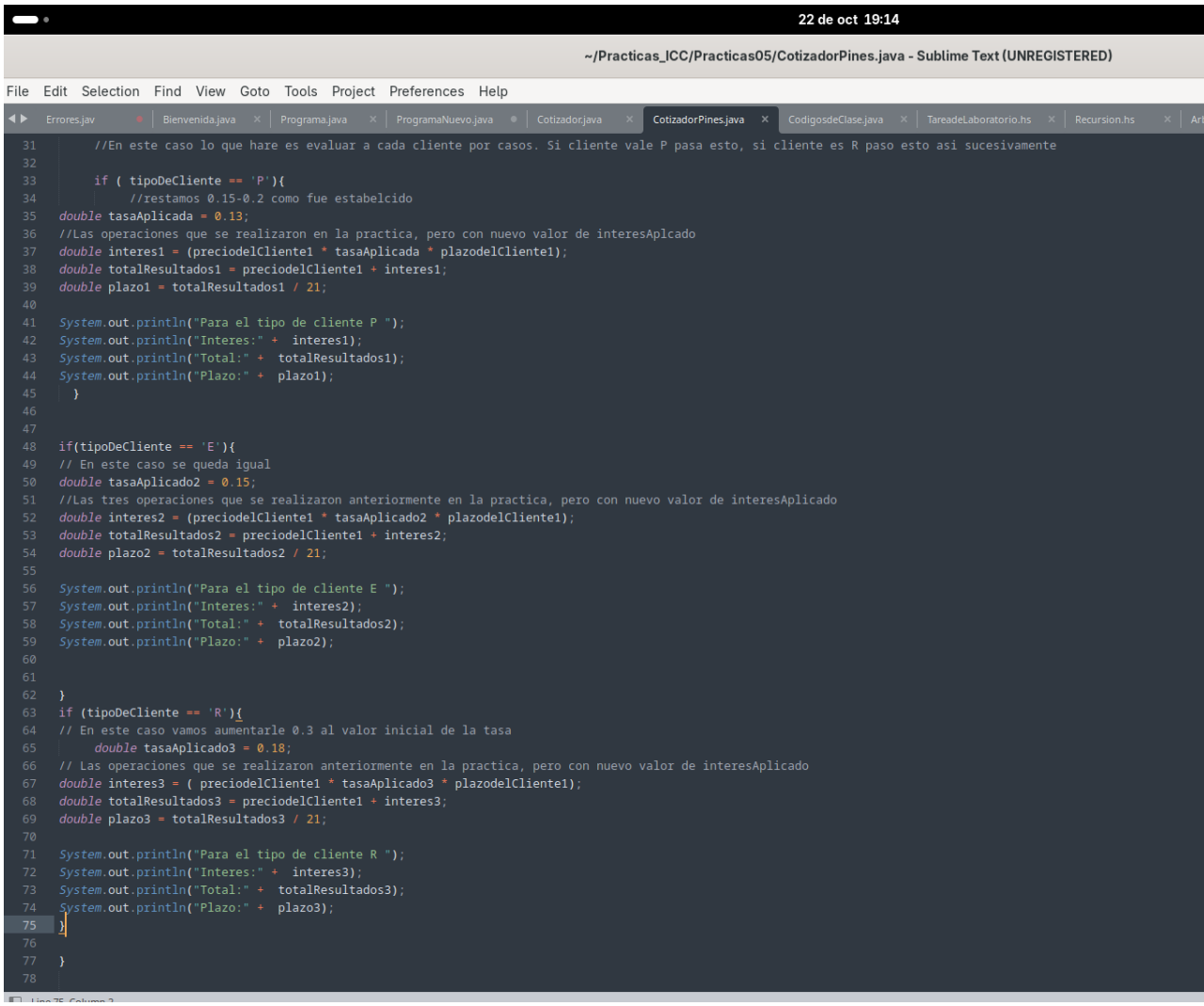


Figure 15: Código del Señor Pines

Compilación y ejecución del Señor Pines

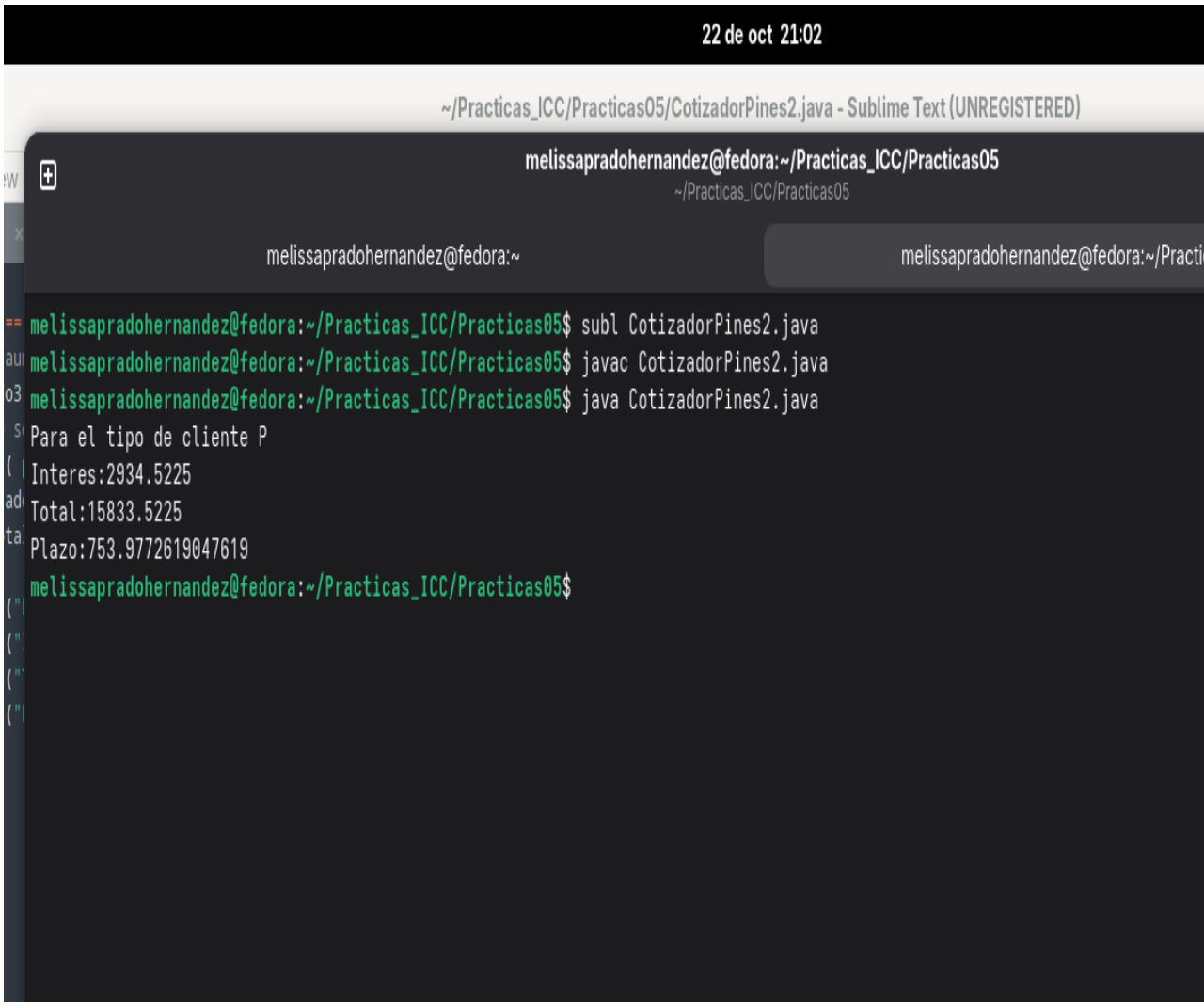


Figure 16: Compilación y ejecución de el Código del Señor Pines

4 Actividad 2.2: Interés Compuesto

Empecé definiendo los datos de la practica anterior como el nombre del cliente nombreDelprimerCliente, el precio preciodelCliente1 = 12899.00, la tasa de interesAnual = 0.15 y el plazo en años plazodelCliente1 = 21.0 / 12.0. Para clasificar al cliente, declaré la variable char tipoDeCliente = 'E'; (probando tambien con 'P' y 'R').
Para validar que no diera errores ni valores negativos, utilicé un if con el operador or revisando preciodelCliente1 ¡= 0 or plazodelCliente1 ¡= 0 or intere-

sAnual ¡= 0. Luego puse un else if con el operador and (tipoDeCliente != 'P' and tipoDeCliente != 'E' and tipoDeCliente != 'R') para asegurar que solo se pudiera acceder si el cliente era P, E o R.
En el bloque else, evalué los casos por separado para ver qué imprimir según la letra del cliente:
Caso Cliente P: Si tipoDeCliente == 'P', asigné la tasa con el descuento tasaAplicada = 0.13, calculé las operaciones simples (interes1, totalResultados1 y plazo1) e imprimí sus resultados.
Caso Cliente E: Si tipoDeCliente == 'E', la tasa se quedó en tasaAplicado2 = 0.15. Calculé primero el interes simple (interes2, totalResultados2, plazo2) e imprimí los datos iniciales junto con los 21 meses de pago y el porcentaje aplicado.
Caso Cliente R: Si tipoDeCliente == 'R', aumenté la tasa a tasaAplicado3 = 0.18, saqué sus operaciones (interes3, totalResultados3, plazo3) y mostré sus valores en pantalla.
Para el tipo de cliente E, calculé la tasa mensual dividiendo la anual entre doce (double tasaMensual = tasaAplicado2 / 12.0;) y declaré double saldoCompuesto = 12899.00;. Empecé a contar desde el mes uno para evitar llegar al mes 22 con la condición int i = 1; i ¡= 21; i++. Adentro del ciclo ocupé la formula para actualizar el saldo (saldoCompuesto = saldoCompuesto * (1.0 + tasaMensual);) e imprimí en cada vuelta el numero de mes con su saldo acumulado.
Al salir del for, saqué la diferencia entre el saldo final y el precio del cliente (double interesCompuesto = saldoCompuesto - preciodelCliente1;) e imprimí los resultados de interes compuesto y saldo compuesto. Por último, usé la herramienta de java Math.pow para calcular preciodelCliente1 * Math.pow(1.0 + tasaMensual, 21) e imprimí resultadoFinal para comprobar que los datos fueran correctos.

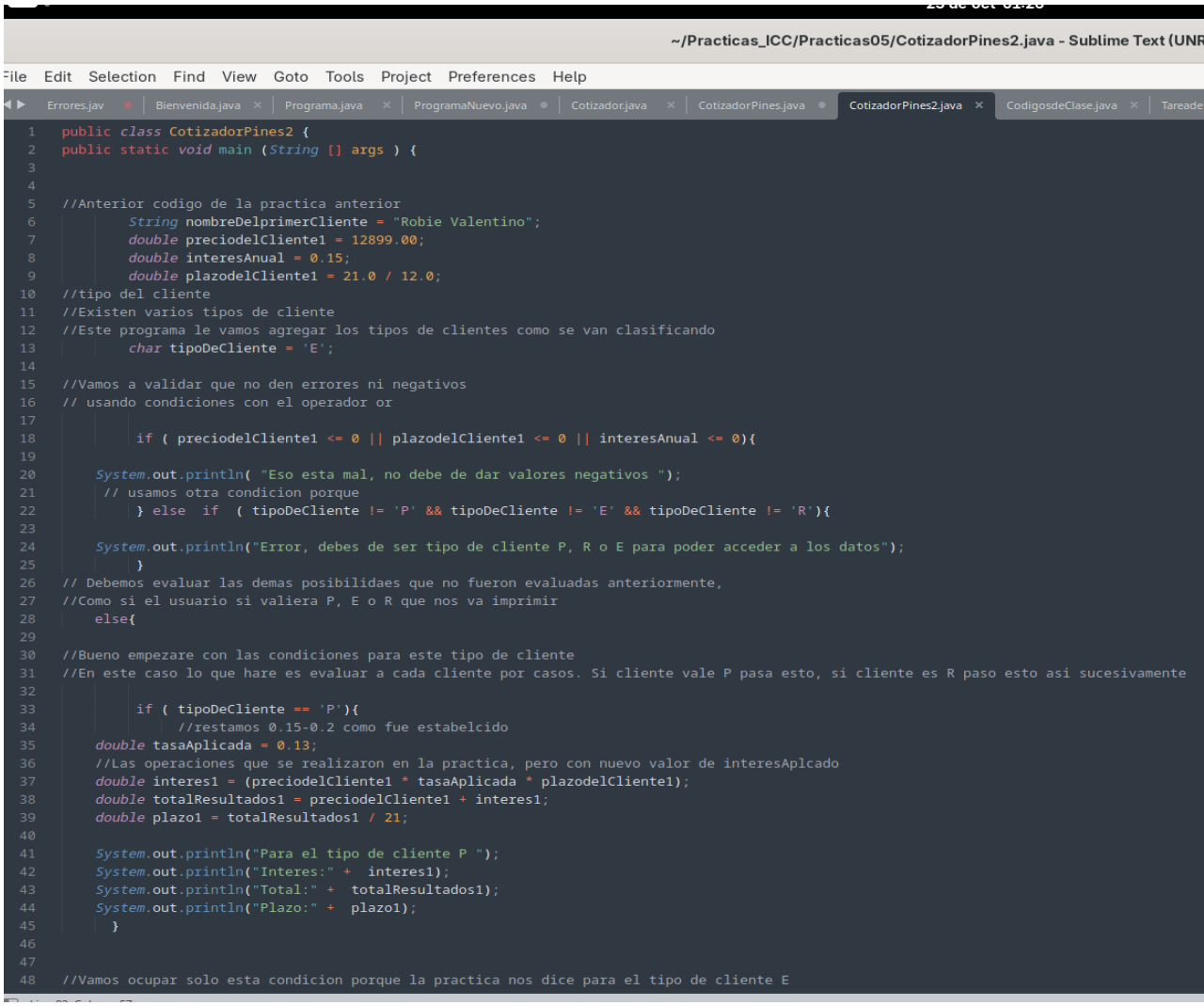


Figure 17: Código del Señor Pines 2

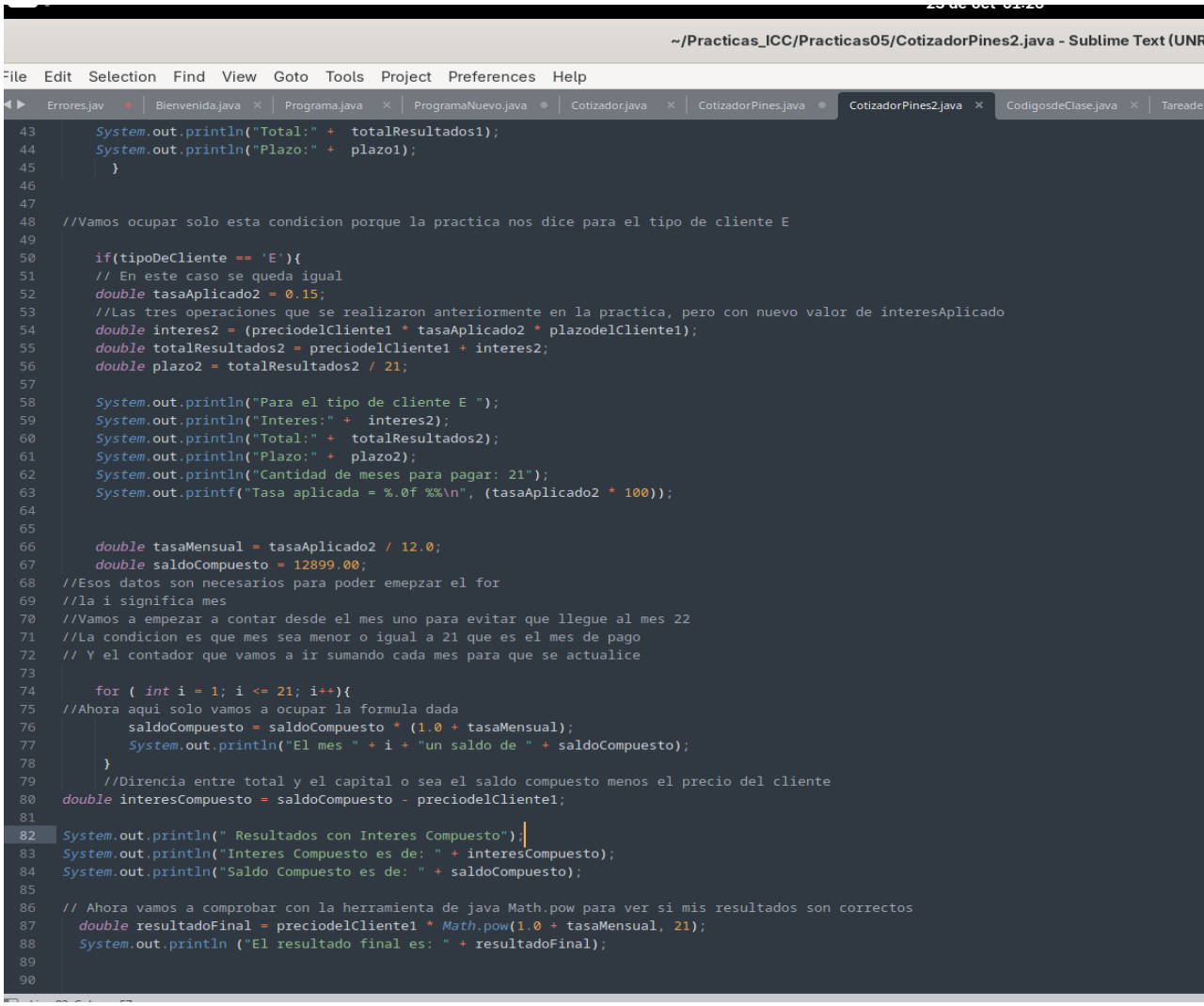


Figure 18: Código del Señor Pines 2

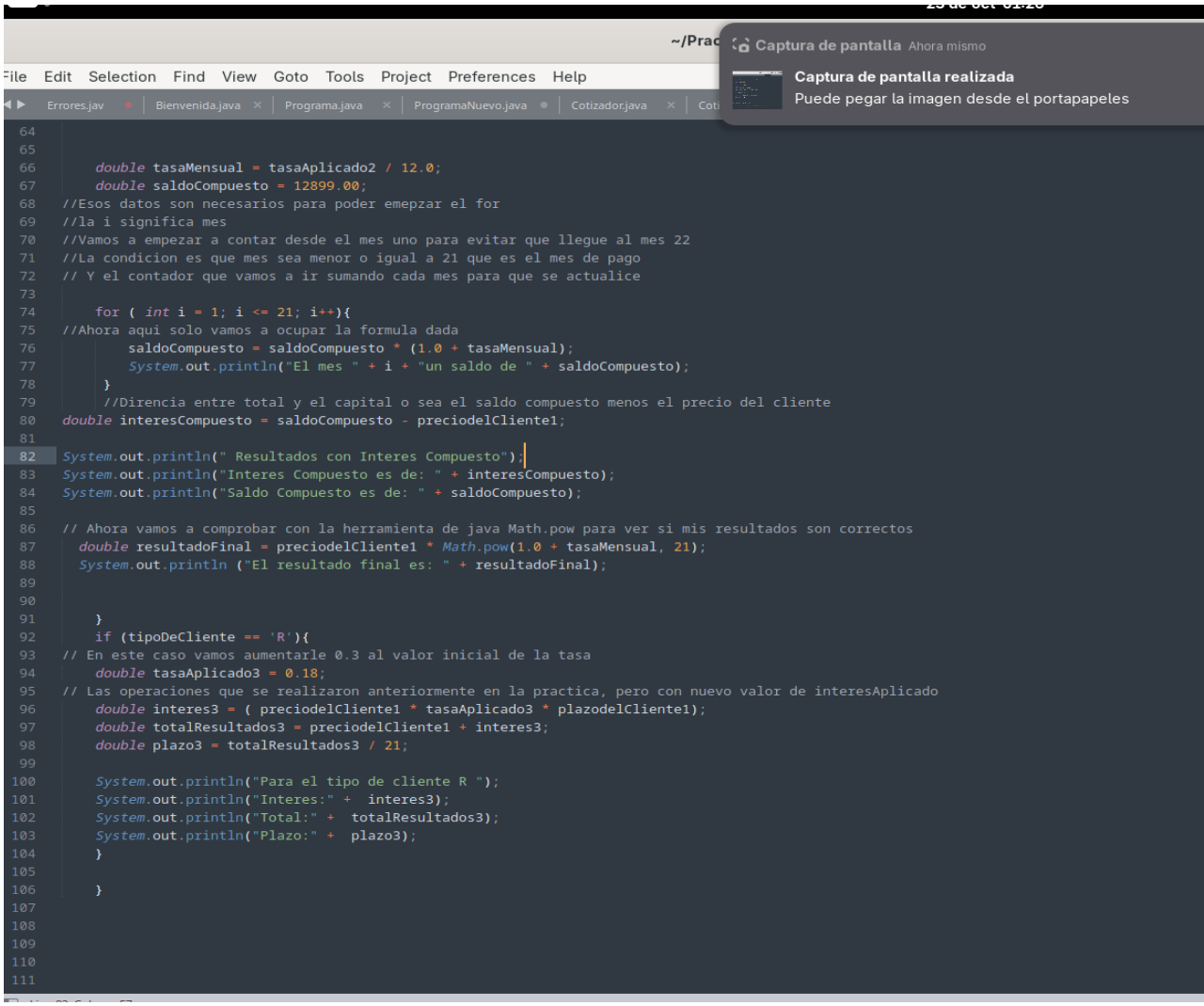


Figure 19: Código del Señor Pines 2

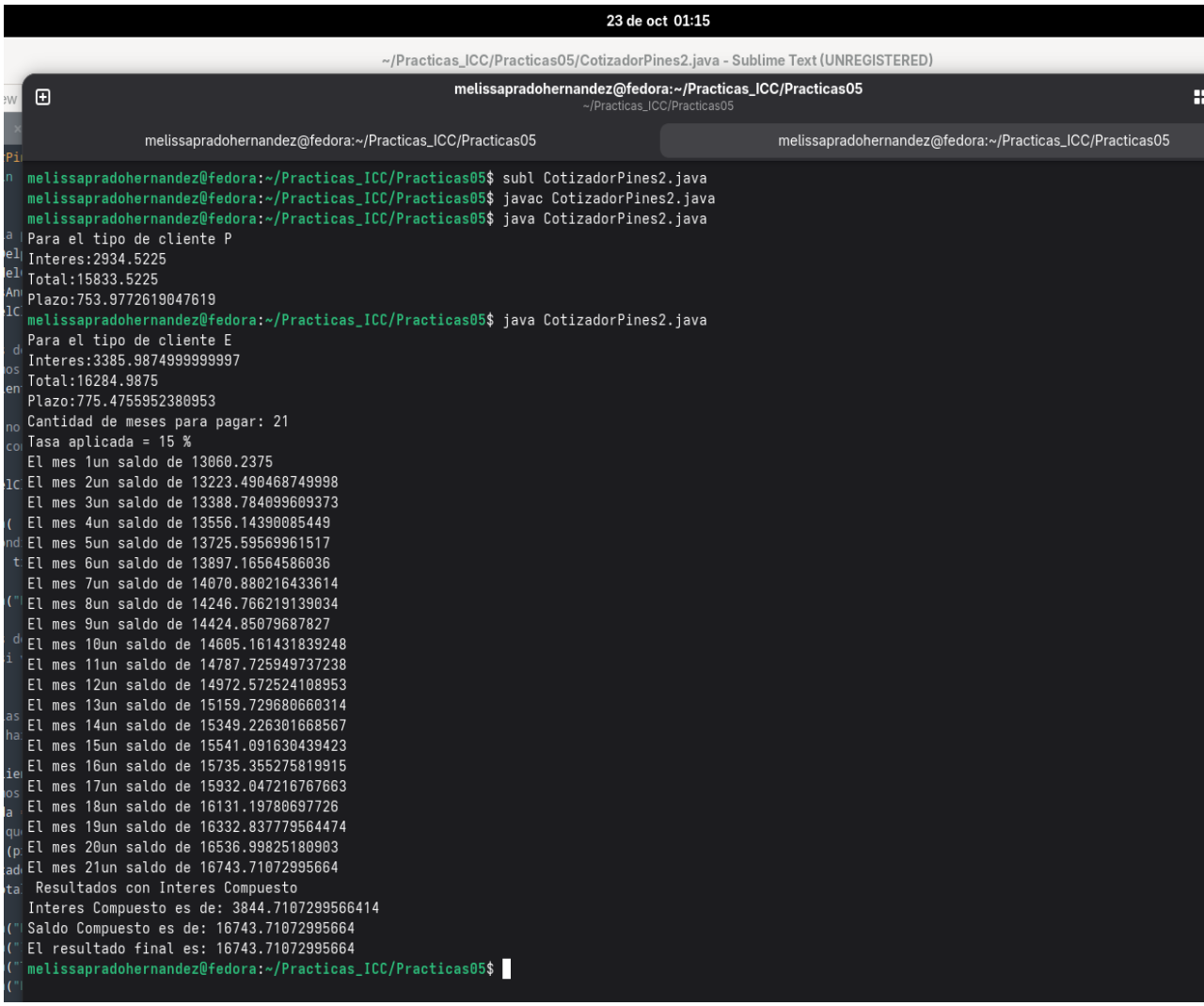


Figure 20: Compilación y ejecución del código del Señor Pines 2